

基于图神经网络的源代码漏洞检测研究综述^{*}

陈子雄¹, 陈旭¹, 景永俊¹, 宋吉飞²

(1. 北方民族大学计算机科学与工程学院, 宁夏 银川 750021; 2. 国家(中卫)新型互联网交换中心, 宁夏 中卫 755000)

摘要:随着开源软件在各个领域的广泛应用,源代码漏洞已经导致了一系列严重的安全问题。鉴于这些漏洞对计算机系统的潜在威胁,检测软件中的源代码漏洞以防止网络攻击已成为一个重要的研究领域。为了实现自动化检测并降低人力成本,研究人员提出了许多基于传统深度学习的方法。然而,这些方法大多将源代码视为自然语言序列而没有充分考虑代码的结构信息,因此其检测效果受到了限制。近年来,基于代码图表示和图神经网络的源代码漏洞检测方法应运而生。全面综述了图神经网络在源代码漏洞检测中的应用,并提出了一个基于图神经网络的源代码漏洞检测通用框架。从文件级别、函数级别和切片级别 3 种漏洞检测粒度出发,系统地总结和阐述了现有的方法和相关数据集。最后,讨论了该领域所面临的挑战,并对未来可能的研究重点进行了展望。

关键词:图神经网络;漏洞检测;数据集;数据流图;控制流图

中图分类号:TP393.08

文献标志码:A

doi:10.3969/j.issn.1007-130X.2024.10.007

A survey of source code vulnerability detection research based on graph neural networks

CHEN Zi-xiong¹, CHEN Xu¹, JING Yong-jun¹, SONG Ji-fei²

(1. School of Computer Science and Engineering, North Minzu University, Yinchuan 750021;

2. National (Zhongwei) New-type Internet Exchange Point, Zhongwei 755000, China)

Abstract: With the widespread application of open-source software across various domains, source code vulnerabilities have led to a series of serious security issues. Given the potential threats these vulnerabilities pose to computer systems, detecting source code vulnerabilities in software to prevent network attacks is a crucial research area. To achieve automated detection and reduce human labor costs, researchers have proposed numerous traditional deep learning-based methods. However, these methods mostly treat source code as natural language sequences and do not adequately consider the structural information of the code, limiting their detection effectiveness. In recent years, methods for detecting source code vulnerabilities based on code graph representation and graph neural networks have emerged. This paper provides a comprehensive review of the application of graph neural networks in source code vulnerability detection and proposes a general framework for source code vulnerability detection based on graph neural networks. Starting from three levels of vulnerability detection granularity: file-level, function-level, and slice-level, the existing methods and relevant datasets are systematically summarized and elucidated. Finally, the challenges faced by this field are discussed, and potential research directions for the future are outlined.

Key words: graph neural networks; vulnerability detection; datasets; data flow graph; control flow graph

^{*} 收稿日期:2024-01-03;修回日期:2024-03-06

基金项目:宁夏回族自治区重点研发项目(2023BDE02017);北方民族大学中央高校基本科研业务费专项资金(2022PT_S04)

通信作者:陈旭(chenxu@nmu.edu.cn)

通信地址:750021 宁夏银川市北方民族大学计算机科学与工程学院

Address: School of Computer Science and Engineering, North Minzu University, Yinchuan 750021, Ningxia, P. R. China

1 引言

随着计算机技术的不断发展,开源组件已经成为现代软件开发的重要组成部分^[1]。在软件开发中,源代码漏洞是一种常见的安全威胁,因此源代码漏洞检测一直是软件开发和信息安全领域的一个重要研究方向^[2]。开源代码具有透明、灵活、可维护、可重用等优点,同时也节省了大量的开发时间和成本,因此被越来越多的企业和个人开发者所采用^[3]。根据 Veracode 公司的研究报告^[4],超过 96% 的产业机构在其开发实现的软件应用代码库中使用开源组件。然而,近 70% 的应用程序都存在开源安全缺陷,这表明开源代码的广泛使用和安全问题已经成为当今软件开发中的一个重要问题。由于代码中可能存在漏洞,这些漏洞可能被攻击者利用,造成严重的安全风险和经济损失。根据国际数据公司 IDG (International Data Group)^[5] 于 2021 年发布的报告,2020 年全球因为软件漏洞导致的经济损失估计超过 5 万亿美元,软件漏洞检测和修复的重要性不言而喻,而这一数字还在不断增长。因此,开源代码的安全问题已经成为一个备受关注的研究领域。

在这样的背景下,开源代码漏洞检测显得尤为重要。通过对开源代码进行全面、深入的漏洞检测,可以及早发现潜在的安全风险,帮助开发人员和组织及时修复漏洞,减少系统被攻击的概率。同时,开源代码漏洞检测也可以提高开源软件的质量和可靠性,增加用户对软件的信任度,促进开源社区的发展和创新。此外,随着开源软件的广泛应用,开源代码漏洞检测也成为组织和企业软件开发和安全管理中必备的环节。因此,开源代码漏洞检测的重要性不容忽视,它为保障软件系统的安全和稳定提供了重要的技术支持。

传统的开源代码漏洞检测方法主要依赖于手工提取代码特征,并利用机器学习模型进行分类。然而,这些方法存在着很多限制,如:难以提取有效特征来应对复杂的代码结构,以及对于新出现的漏洞难以实现准确的分类等。近年来,基于深度学习的方法已经成为开源代码漏洞检测的主要研究方向之一^[6-10]。特别地,图神经网络 GNNs (Graph Neural Networks) 作为一种强大的图表示学习工具,已经被广泛应用于图数据的分析和处理^[11]。GNNs 能够学习和表示复杂的非欧几里得数据,通过将源代码表示为图,可以从源代码中提取出程序

的结构信息,从而实现对源代码漏洞的精准检测^[12-17]。因此,越来越多的研究人员开始探索使用 GNNs 进行开源代码漏洞检测。

本文围绕基于图神经网络的源代码漏洞检测技术展开研究和讨论,其组织结构如下:第 2 节主要从开源代码漏洞检测、图神经网络和图数据建模 3 个部分对相关预备知识和概念进行介绍及定义;第 3 节从漏洞检测粒度出发,分别从文件级别、函数级别以及切片级别对基于 GNNs 的源代码漏洞检测技术进行讨论、分析和总结;第 4 节主要对源代码漏洞数据集进行总结和分析;第 5 节对基于图神经网络的源代码漏洞检测领域的现存挑战和未来方向进行了讨论和分析;最后,第 6 节对本文进行了总结。

2 预备知识

本节分别介绍了开源代码漏洞检测、图神经网络和图数据建模 3 部分的相关知识。在开源代码漏洞检测部分介绍了开源代码漏洞检测技术在机器学习领域的发展历程;在图神经网络部分主要介绍了图神经网络的一些相关知识;在图数据建模部分主要介绍了将源代码表示为图结构的一些基本方法。

2.1 开源代码漏洞检测

开源代码漏洞检测技术是一种帮助开发人员在开发过程中尽早发现代码问题的重要技术。它通过分析源代码来识别潜在的代码缺陷和安全漏洞,从而帮助开发人员在代码提交到生产环境之前尽早发现并解决问题。

开源代码漏洞检测技术在机器学习领域的发展经历了传统机器学习方法、深度学习方法以及新兴的基于图神经网络的方法。

基于机器学习的开源代码漏洞检测方法主要采用传统的特征提取和分类器技术^[18-23]。这些方法通常使用专家人工构造的特征,如代码语法特征、统计特征和代码结构特征,然后使用分类器如支持向量机 SVM (Support Vector Machine)^[24]、随机森林 (Random Forest)^[25] 等进行漏洞分类。这些方法的主要优点是易于实现和解释,但受限于手工特征的代表能力和泛化能力。

随着深度学习在计算机视觉和自然语言处理等领域的成功应用,研究人员开始将深度学习方法应用于开源代码漏洞检测。深度学习方法利用神经网络模型从源代码中自动学习表示。例如,使用

卷积神经网络 CNNs(Convolutional Neural Networks)^[26]对源代码进行特征提取,或者使用循环神经网络 RNNs(Recurrent Neural Networks)^[27]对代码序列进行建模。深度学习方法能够学习更丰富的特征表示,但在处理图结构和复杂依赖关系方面存在一定的局限性。

近年来,基于图神经网络的方法引入了图表示来处理源代码漏洞检测任务。这些方法将源代码表示为图,并利用图神经网络模型对构造的代码图进行学习和推理。例如,图卷积网络 GCN(Graph Convolutional Network)^[28]和图注意力网络 GAT(Graph Attention Network)^[29]可以在图结构中捕获节点之间的依赖关系和上下文信息。这种方法能够更好地建模代码中的复杂关系,并在源代码漏洞检测中取得了显著的性能提升。现在,越来越多的研究人员开始探索使用 GNNs 进行源代码漏洞检测的研究。

2.2 图神经网络

首先,介绍图的计算任务;其次,介绍图神经网络的基本原理;最后,对一些经典的图神经网络模型进行简要介绍。

2.2.1 图的计算任务

图上有各种各样的计算任务,这些任务主要分为 2 大类:一类侧重于节点的任务(Node-focused tasks),其中整个数据通常表示为一个图,节点表示数据样本;另一类侧重于图的任务(Graph-focused tasks),其中数据往往包括一组图,每个数据样本是一个图。本节将简要介绍每个类别的代表性任务。

(1)侧重于节点的任务。

侧重于节点的任务已被广泛研究,如节点分类、节点排序、链接预测和社区检测。

在现实世界中,节点常常与重要的信息相关,且这些信息可以被视为这些节点的标签,例如,在社交网络中,这类信息可以是用户的人口统计属性,如年龄、性别、职业、兴趣和爱好。这些标签通常有助于描述该节点的特征,并可用于许多重要的应用,例如,Facebook 等社交网络,可以利用与兴趣爱好相关的标签向用户推荐相关内容(如新闻和事件)。然而在现实中,通常很难为所有节点获得完整的标签集,例如,只有不到 1% 的 Facebook 用户提供了完整的个人属性。因此,多数时候很可能得到一个只有一部分节点有标签的图,而那些无标签的节点就需要通过模型预测标签。节点分类的

定义如下:

定义 1(节点分类) 用 $G = \{v, \epsilon\}$ 表示一个图,其中 v 是节点集, ϵ 是边集。 v 中的一部分节点有标签,记为 $v_l \subset v$,剩下的节点没有标签,记为 v_u 。由此可知, $v_l \cup v_u = v, v_l \cap v_u = \emptyset$ 。节点分类的目标是利用图 G 和 v_l 的标签信息学习一个映射 φ ,映射 φ 可以预测无标签节点 v_u 的标签。

基于图神经网络的源代码漏洞检测任务主要是侧重于节点的任务。尽管在源代码漏洞检测中构建了图结构,但主要关注的是图中节点(即代码的不同元素,如函数、语句和变量等)的表示学习和漏洞检测。图神经网络的目标是学习每个节点的表示向量,以捕捉节点之间的关系和特征,从而识别和预测节点级别的漏洞。

在这种任务中,图神经网络的输入是源代码中的节点(代码元素)和边(节点之间的关系),并通过图神经网络的层次结构和学习算法,对每个节点进行表示学习。通过传递和更新节点的特征向量,网络可以学习到节点的丰富表示,以便进行漏洞检测和分类预测。

(2)侧重于图的任务。

在现实世界中,存在许多种侧重于图的任务,如图分类、图匹配和图生成。接下来,介绍最具代表性的侧重于图的分类任务——图分类。

节点分类将图中的每个节点视为一个数据样本,旨在为这些未标记的节点分配标签。而在另外一些应用中,每个样本都可以表示为一个图。例如,在化学信息学中,化学分子可以用图表示,其中原子是节点,它们之间的化学键是边;这些化学分子可能有不同的性质,如溶解度和毒性,可以作为其标签处理。在实际应用中,可能需要自动预测新发现的化学分子的性质,这一目标可以通过图分类的任务来实现,其本质是预测未标记图的标签。由于图结构的复杂性,传统的分类方法无法实现图的分类,所以图分类的研究是极具挑战性的。图分类的定义如下:

定义 2(图分类) 给定一组带标签的图数据集 $D = \{(G_i, y_i)\}$,其中 y_i 是图 G_i 的标签,图分类的目标是在数据集 D 上学习一个映射函数 f ,用以预测未标记图的标签。

虽然基于图神经网络的源代码漏洞检测任务主要是侧重于节点的任务,但是也有一部分基于图神经网络的源代码漏洞检测任务是侧重于图的任务。在这种情况下,将整个源代码视为一个图结构,其中节点表示源代码中的元素,边表示元素之

间的关系;任务的目标是对整个图进行分类,即判断整个源代码是否含有漏洞。在这种情况下,就需要考虑整个图的全局结构和节点之间的关联,以便进行准确的图级别分类。

这种侧重于图的任务可能会涉及对整个图的聚合和汇总,以获得图级别的特征表示。例如,可以使用图池化操作将整个图的信息汇集到一个固定长度的向量中,然后进行图级别的分类预测。

源代码漏洞任务检测时需要根据具体的任务定义和研究设置来确定是侧重于节点的任务还是侧重于图的任务。有时候,源代码漏洞检测可能会涉及对节点和图 2 个层面的建模和预测,以综合考虑局部和全局信息的影响。因此,在基于图神经网络的源代码漏洞检测任务中,节点级别和图级别的任务都有其重要性和应用价值。

2.2.2 图神经网络的基本原理

图神经网络的概念最早由 Gori 等人^[30]提出,并由 Scarselli 等人^[31]进一步阐明。图神经网络是一类专门用于处理图结构数据的神经网络模型,它通过利用图的拓扑结构和节点之间的关系来进行特征提取和信息传播,具有以下基本原理:

(1)图的表示和更新:图神经网络中的基本单位是节点和边。每个节点表示一个实体或对象,边表示节点之间的关系或连接。节点和边均被赋予特征向量,这些向量捕获了各自的属性信息。通过迭代地更新节点的局部与全局信息,对节点的局部信息和全局信息进行聚合和整合,从而提升图神经网络的表示能力。

(2)层次化的信息传播:图神经网络的核心操作是信息传播。在每一层中,节点的表示向量会与其邻居节点的表示向量进行交互和聚合,从而获取邻居节点的信息。这种层次化的信息传播能够将节点的局部信息和全局信息进行交互和融合,从而更好地捕捉图的结构和关系。

(3)节点嵌入和图嵌入:图神经网络可以通过节点嵌入和图嵌入来表示整个图的特征。节点嵌入是指将每个节点映射到一个低维空间中的向量表示,反映节点的语义和特征。图嵌入是指将整个图映射到一个低维向量表示,反映图的整体结构和特征。

2.2.3 经典的图神经网络模型

(1)GCNs。

图卷积网络是图神经网络中最为经典的模型之一,由 Kipf 等人^[28]在 2016 年提出。它是一种

基于图数据的深度学习模型,在处理图结构数据时具有优秀的表征学习能力。GCN 通过在图的节点之间传递和聚合信息来学习节点的特征。它采用类似于传统卷积神经网络的思想,在图上定义了一种卷积操作,使节点可以利用其邻居节点的信息进行特征更新和传播。GCN 的核心思想是将节点的特征与其邻居节点的特征进行加权平均,从而得到更新后的节点表征。这样的操作能够有效地利用图结构中的信息,使节点的特征能够同时考虑到自身特征和邻居节点的特征。GCN 在图数据的节点分类、链接预测、图分类等任务中取得了很好的效果,并且在许多领域中展现出了潜力。

(2)GAT。

GAT 是一种基于注意力机制的图神经网络,由 Veličković 等人^[29]在 2018 年提出。它通过引入注意力机制,使节点能够动态地分配不同邻居节点的权重,并根据这些权重来更新自身的表征。这种机制使得 GAT 能够学习到节点之间的复杂非线性关系,并在处理图数据的节点分类、链接预测和图分类等任务中表现出色。GAT 在多个领域中展现出潜力,如社交网络分析、推荐系统和生物信息学等,为图数据分析提供了一种强大而灵活的工具。

(3)GraphSAGE(Graph SAmple and aggregate)。

GraphSAGE^[32]是一种基于采样的图神经网络,它的核心思想是利用邻居节点的信息来聚合目标节点的特征表示。通过采样邻居节点子集并进行聚合操作,GraphSAGE 能够对每个节点生成一个固定长度的特征向量。具体来说,GraphSAGE 包括 2 个关键的步骤:采样和聚合。首先,模型对每个节点进行邻居采样,即从每个节点的邻居中随机选择一部分进行采样。然后,对于每个节点,采样得到的邻居节点特征被聚合到目标节点的特征表示中,通过聚合操作(如平均池化、最大池化等)得到节点的最终表示。GraphSAGE 可以用于节点分类和链接预测等任务。例如,在社交网络分析中,它可用于社群检测;在生物信息学领域,它可用于分析生物网络和蛋白质相互作用。这些应用展示了 GraphSAGE 作为图数据分析的强大工具的潜力。

(4)GGNNs(Gated Graph Neural Networks)。

GGNNs^[33]是一种具有门控机制的图神经网络,其核心思想是利用循环神经网络的思想对图数

据进行序列化处理。它通过在节点之间进行信息传播,逐步更新节点的表示,从而获得节点的最终特征表示。GGNNs 使用了门控机制来控制节点之间的信息传递,包括更新门和重置门,以动态地调整节点的更新程度和信息交互方式。GGNNs 可以对图中的节点进行序列化处理,从而提高模型的表达能力和准确性。GGNNs 可以用于节点分类、图级别的分类和预测任务。例如,它可以用于社交网络中预测用户的兴趣,用于药物发现中预测分子的属性,等。

总的来说,以上几个模型都是图神经网络领域的经典模型,它们都在不同的任务和应用场景中得到了广泛的应用和研究。图神经网络已经成为处理图结构数据的重要工具之一,不同的图神经网络模型可以在不同的任务中发挥出其优势和特点。这些图神经网络模型在源代码漏洞检测中的应用主要集中在学习源代码的表示、提取特征、发现漏洞模式、异常检测和漏洞预测等任务上。它们通过有效地利用图结构数据的信息,提供了一种新颖且有效的方法来改进源代码漏洞检测的性能和准确性。

2.3 图数据建模

源代码通常具有复杂的结构和关联关系。将代码表示为图可以保留代码中的层次结构、依赖关系和调用关系等信息,从而更好地捕获代码的结构特征。漏洞的存在通常与代码的上下文有关,例如代码中的函数调用、变量定义等。通过将源代码表示为图,图神经网络可以学习到节点之间的关系和上下文信息,更好地捕捉潜在的漏洞特征。

基于图神经网络的漏洞检测方法在不同粒度(文件级别、函数级别和切片级别)下的数据建模存在以下差异:

(1)文件级别数据建模:在文件级别的漏洞检测中,整个源代码文件被看作一个单一的图结构。数据建模过程将源代码文件转换为一个图,其中节点表示源代码文件中的不同代码元素(例如函数、变量和常量等),边表示这些代码元素之间的关系(例如函数调用关系和变量引用关系等)。节点和边上包含的特征信息可以是代码元素的属性(例如名称、类型)或其他相关信息。

(2)函数级别数据建模:在函数级别的漏洞检测中,每个函数被看作一个独立的图结构。数据建模过程将每个函数的源代码转换为一个图,其中节点表示函数内的代码元素,边表示代码元素之间的关系。与文件级别类似,节点和边可以携带代码元

素的特征信息。

(3)切片级别数据建模:在切片级别的漏洞检测中,关注的是与特定语句相关的代码切片。数据建模过程首先确定要提取的切片,然后将切片的源代码转换为一个图结构。图的节点表示切片中的代码元素,边表示代码元素之间的关系。同样,节点和边可以包含代码元素的特征信息。

基于图神经网络的漏洞检测需要将源代码首先构造成图结构,下面介绍几种基本的将代码表示为图结构的类型:

(1)抽象语法树 AST(Abstract Syntax Tree)^[34,35]:是一种由编译器生成的树状结构,它反映了程序的语法结构和操作流程。AST 可以用于代码分析、优化和转换,以及可视化程序结构。AST 表示源代码的语义和句法特征,已被用于不同的软件工程任务,例如变量名称预测^[36]、函数名称预测^[37]、代码摘要^[37]、代码字幕^[37]、函数分类^[38]、类型预测^[39]和错误预测^[40]。

(2)控制流图 CFG(Control Flow Graph)^[41,42]:是一种描述程序执行路径的图形表示,它显示了程序中的基本块、条件语句、循环和跳转操作。CFG 可以用于代码分析、测试和优化,以及可视化程序的控制流。

(3)数据流图 DFG(Data Flow Graph)^[36]:是一种描述程序中数据流动的图形表示,它显示了数据在程序中的流动路径和转换。DFG 可以用于识别和解决数据依赖性问题,以及可视化程序的数据流动。

(4)代码复合图 CCG(Code Composite Graph):是由抽象语法树、控制流图和数据流图组成的综合图结构。它将这 3 种图结构的信息融合在一起,提供了更全面、更丰富的代码表示方式。如图 1 所示,CCG 由一组 AST 的节点和不同类型的边组成。语句和谓词节点通过 CFG 和 DFG 连接。因此,边的类型反映了不同节点之间的语法和语义关系。

(5)程序依赖图 PDG(Program Dependence Graph)^[43]:是一种描述程序元素之间依赖关系的图形表示,表示程序中的控制流、数据流和调用关系等依赖关系。PDG 的节点代表程序中的语句或表达式,边表示不同语句之间的依赖关系,如控制依赖、数据依赖和调用依赖。通过构建 PDG,可以全面了解程序的依赖结构,为程序分析和优化提供有价值的信息。在源代码漏洞检测中,PDG 被广泛应用于表示源代码的结构和依

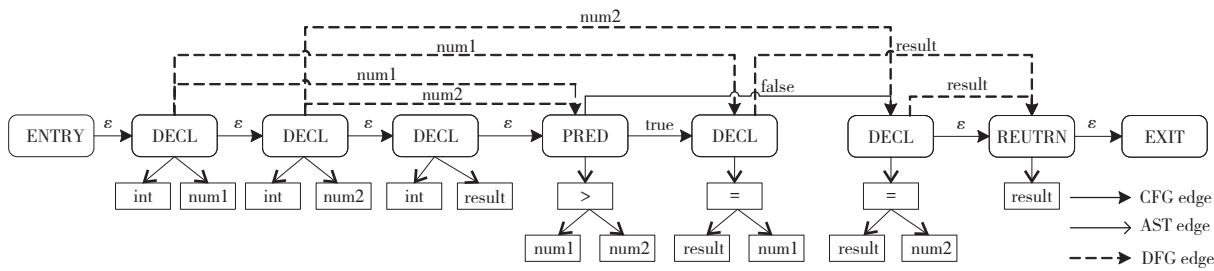


Figure 1 Code composite graph

图 1 代码复合图

赖关系,为图神经网络提供输入数据,实现准确的漏洞检测和预测。

(6)代码属性图 CPG(Code Property Graph)^[36]: 它将经典程序分析的概念(即 AST、CFG 和 PDG)合并到一个联合数据结构中,是一种用于表示程序代码的图形表示法。它将程序源代码转换为一种通用的图形形式,便于进行静态分析、代码检查、漏洞检测等操作。CPG 通常包括代码的语法结构、控制流、数据流、依赖关系和类型信息等属性。

表 1 对上面介绍的将代码表示为图结构的几种类型进行了优劣对比。由表 1 发现,将代码表示为单一图,简化了输入数据,适合于特定类型的漏洞检测,其优势是模型训练和推理速度较快,但劣势是可能丢失其他类型信息;将代码表示为复合图,综合了多种关系,考虑了更多的上下文信息,适合于综合性的漏洞检测,其优势是可以更全面地捕获代码关系,但劣势是计算复杂度较高。

源代码可以表示为各种各样的图,具体要结合研究情况进行构图。研究人员可以将源代码构造为单一的数据流图 DFG 或控制流图 CFG 作为图神经网络的输入,也可以将 AST 与二者相结合进行构图,不同的图所能覆盖的源代码信息不同,图越复杂覆盖信息越多,但图神经网络的计算难度也

呈指数增加。因此,要基于自己的研究需要对源代码进行构图表示。

3 基于 GNN 的源代码漏洞检测技术

目前已经有很多基于深度学习的源代码漏洞预测模型,但程序语言不同于自然语言,其更具有结构性与层次性,使用神经网络进行漏洞检测的方案大多基于传统自然语言处理的思路,将源代码当作序列样本处理,忽视了代码中所具有的结构特征,从而遗漏了可能存在的漏洞。对于源代码的表示本文在进行了大量的研究后发现,基于图的表示方法更适合源代码,是一种更自然的代码表示。将源代码表示为图封装了源代码丰富的语义和结构信息,图神经网络与代码的各种属性图可以更好地适配,因此逐渐成为研究人员在漏洞检测任务上的热点模型。

本节将介绍近年来图神经网络在源代码漏洞检测上的研究情况。首先,介绍本文提出的基于 GNN 的源代码漏洞检测通用框架;其次,基于文件级别、函数级别和切片级别 3 种漏洞检测粒度介绍 GNN 在源代码漏洞检测上的研究情况;最后进行小结分析。

Table 1 Code representation diagram

表 1 代码表示图

图的类型	图表示	优势	劣势
单一图	AST	能够捕获源代码的语法结构,用于检测基于语法规则的漏洞	缺少详细的上下文信息,无法很好地捕获语义关系
	CFG	展示程序中的控制流程,有助于检测与代码逻辑相关的漏洞	不包括数据流信息,无法充分捕获数据依赖关系
	DFG	突出数据传递和变量之间的依赖关系,用于检测与数据流相关的漏洞	忽略控制流信息,可能无法完整反映代码逻辑
复合图	CCG	将多种图表示整合为复合图,综合考虑语法、控制流、数据流等信息	复杂度高,可能需要更复杂的图神经网络模型
	PDG	结合控制和数据依赖信息,能够捕获综合的代码关系	生成过程较复杂,可能对计算资源和时间成本要求较高
	CPG	综合了代码的属性信息,有助于捕获代码中的语义信息	可能需要特定的预处理来提取属性信息

3.1 GNNvulD 框架

本文经调查研究发现不同的基于 GNN 的源代码漏洞检测技术存在相同的 4 步设计流程,不同方法设计不同之处也体现在这 4 步。因此,基于现有的关于图神经网络在源代码漏洞检测上的应用研究,本文设计了一个通用框架 GNNvulD,此框架主要描述了将 GNN 模型应用在源代码漏洞检测上的通用设计步骤,如图 2 所示。

基于图神经网络的源代码漏洞检测通用框架 GNNvulD 分为以下 4 个步骤:

Step 1 收集源代码漏洞相关数据。利用公开的漏洞数据库如 CVE(Common Vulnerabilities and Exposures)和 NVD(National Vulnerability Database)获取已知的漏洞信息。同时,可以从 GitHub 等开源代码库中获取源代码文件,包括存在漏洞和无漏洞的代码样本。

Step 2 将源代码文件转换为图结构。根据代码的结构和依赖关系构建图,常见的图表示方法包括 CFG^[41,42]和 DFG^[36]等。一个文件、函数或代码片段都可以表示为一个图,图中的每个节点表示代码片段、函数或语句,边表示代码之间的控制流或数据依赖关系。图数据建模可参见 2.3 节的相关介绍。

Step 3 利用 GNN 模型提取特征和学习。GNN 模型通过节点和边的信息来学习代码图的表示,捕捉代码之间的关系和特征。常用的 GNN 模型包括 GCN^[28]、GraphSAGE^[32]、GAT^[29]等。这些模型通过多层图卷积操作来逐步聚合和更新节点的特征。

Step 4 对源代码进行训练和分类预测。利

用特征提取后的向量表示对源代码进行训练和分类预测。可以使用监督学习算法(如支持向量机 SVM^[24])或深度学习模型(如多层感知机 MLP(Multi-Layer Perceptron))来构建分类器。训练过程中,将已知是否存在漏洞的源代码样本作为标签进行模型训练,然后对未知样本进行预测,判断其是否存在漏洞。

3.2 文件级别

基于图神经网络的源代码漏洞检测可以在文件级别进行检测,这意味着它关注的是整个源代码文件的漏洞情况。在 GNNvulD 框架中的 Step 1 收集和整理源代码数据集阶段,包括包含多个文件的软件项目,确保数据集中包含正常代码和已知漏洞的代码示例。在 Step 2 构建代码图阶段,将整个源代码文件视为一个图,其中每个节点代表一个代码文件中的元素(如函数、类、变量等)。节点之间的边表示不同元素之间的关系(如函数之间的调用关系)。

在漏洞检测中,一些漏洞是由非本地代码片段的交互造成的,可能需要整个文件甚至更多文件才能识别,这就需要使用文件级别的漏洞检测技术。Rabheru 等人^[44]介绍了 DeepTective,这是一种检测 PHP(HyPertext Preprocessor)源代码漏洞的深度学习方法。DeepTective 在函数和文件级别的粒度上专注于检测源代码中的 SQLi^[45]、XSS^[46]和 OSCi^[47]漏洞。DeepTective 在 Step 1 使用来自 SARD 和 GitHub 中的代码构造漏洞代码数据样本集合,使用 PHP 漏洞测试套件生成器^[35]生成了详细的测试用例;在 Step 2 使用 joernPHP^[29]从样本中解析和提取 CFG;在 Step 3 将门控循环单元

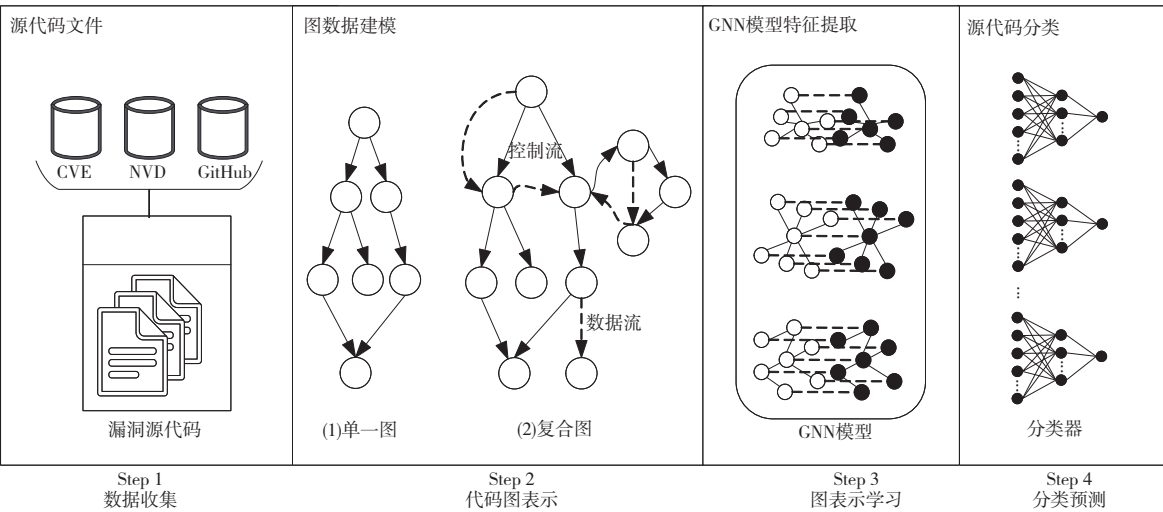


Figure 2 GNNvulD Framework
图 2 GNNvulD 框架

GRU(Gate Recurrent Unit)和 GCN 结合在一个新颖的混合架构中,使用 GRU 来学习源代码标记的长期顺序依赖关系,使用 GCN 合并来自 CFG 的上下文信息。这样做能够利用这 2 种技术的优势,将句法、结构和语义信息嵌入到机器学习模型中。DeepTective 展现出的可扩展的执行性能,可以处理大型源代码库,并在合成数据集和实际数据集上实现比现有最先进技术更好的分类性能。在 Step 4 中 DeepTective 在合成数据集上实现了近乎完美的分类,在现实数据集上的 $F1$ 得分为 88.12%,优于其他相关方法。DeepTective 专注于检测源代码中的 SQLi、XSS 和 OSCI 漏洞,尽管这些漏洞在网络安全中具有重要意义,但它们只代表了源代码中潜在漏洞的一小部分。因此,DeepTective 在检测漏洞多样性方面存在局限性,无法涵盖其他类型的漏洞,如逻辑漏洞、权限问题和信息泄漏等。

基于文件粒度的漏洞检测可以有效地进行跨项目漏洞分析。DeepTective 的训练过程和模型可能过于特定,难以应对不同项目之间的差异性,因此在这一方面存在缺陷。跨项目漏洞分析需要更具灵活性和通用性的方法,以确保模型能够有效地检测各种项目中的漏洞,而不仅仅局限于特定项目或漏洞类型。Ghaffarian 等人^[48]开发了一个名为 PROGEX(PROgram Graph EXtractor)的跨平台工具,它能够高效地分析代码库并高速提取图形,能够以合适的格式提取程序源代码文件的各种图形表示,例如 AST、CFG 和 PDG。他们设计了一种定制的程序中间图表示,在 Step 2 使用定制的 AST、CFG、PDG 和 DDG 作为信息来源。在此基础上,本文对节点和边的文本信息(变量名、类型)使用 TF-IDF^[49]进行向量化;在 Step 3 图模型的选择上,其选用了 GCN 与 GAT 进行对比实验。实验结果显示:与 GCN 和所选基线方法相比,GAT 模型具有更好的泛化能力,并且显示出作为跨项目漏洞检测可接受的解决方案的主要潜力。另一方面,GCN 似乎很容易过度拟合,并且似乎更适合项目内的漏洞检测。

文件级别的检测粒度具有综合性和整体性的特点。基于图神经网络的源代码漏洞检测方法在文件级别的检测粒度可以提供对整个源代码文件的综合分析,能够捕捉到文件级别的漏洞模式和关联性。其次,文件级别的检测可以帮助快速定位和修复漏洞,减少漏洞在代码库中的传播和影响,但在定位精确性和细粒度检测方面存在一些限制。

3.3 函数级别

基于函数级别的图神经网络漏洞检测在 GNNVulD 中的 Step 1 收集和整理源代码数据集阶段,将获得的文件数据按照函数划分为单个样本,包括包含多个函数的软件项目,确保数据集中包含正常代码和已知漏洞的代码示例。在 Step 2 构建代码图阶段,将函数内的代码表示为一个图,其中每个节点代表函数中的一个代码语句或表达式,边表示代码语句之间的控制流和数据流关系。

因为漏洞数据往往是敏感的,难以获取,且漏洞样本数量有限,因此在收集源代码漏洞数据集阶段,获取高质量和大规模的漏洞代码数据集是一个挑战。为了提供足够的训练数据来构建有效的深度学习模型,补充常见漏洞披露 CVE 和软件保障参考数据集 SARD 中的可用代码示例,Wang 等人^[50]开发了一个自动数据收集框架 FUNDED(Flow-sensitive vUINerability coDE Detection),从 GitHub 等开源存储库收集漏洞的代码样本。这弥补了现实生活中漏洞代码样本短缺的现状,使得用于模型训练的数据集更加强大大。

GNNVulD 框架通常在 Step 1 阶段完成漏洞代码数据集的收集之后便进入 Step 2 阶段,但这样存在一定的局限性,即训练的模型不兼容不同类型的编程语言。基于此问题,陈皓等人^[51]在 Step 2 源代码构图之前,将源代码转换为中间语言,然后使用转换得到的中间语言构图不仅可以屏蔽不同类型高级语言的语法差异、兼容不同类型的高级语言,也能起到扩展训练样本的作用。他们采用 LLVM IR(Low Level Virtual Machine Intermediate Representation)的中间语言方案将源代码预编译为中间表示,使得检测方法能够兼容不同类型的编程语言。通过 Clang 编译器将数据集代码样本编译为可读形式的中间语言代码样本,在 Step 2 阶段将中间代码表示构造成控制流图 CFG,在 Step 3 阶段使用提出的由多组图卷积 GCN 和池化层组成基本单元的图神经网络来完成在图级别上的端到端分类学习问题;最后在 Step 4 阶段通过应用多层感知机和 Softmax 层获得最终的漏洞预测标签。其中 LLVM IR 代码的控制流图抽取采用 LLVM opt 工具^[52]实现;采用 word2vec 算法^[53]实现控制流图基本块初始化向量的预训练;采用 PyG(Pytorch_Geometric)框架^[54]进行神经网络模型开发和训练。与传统的 LSTM 网络相比,陈皓等人^[51]提出的方法效果更优,但该方法也可以与较新的一些深度学习模型相结合,以进一步

挖掘其模型的潜在性能。

源代码漏洞检测利用图神经网络方法时,需要将源代码转换为图形表示。然而,在 Step 2 阶段转换过程中如何保留源代码丰富的句法、语义和结构信息,并同时保持高效的计算效率,成为一个挑战。

针对这个问题,Wu 等人^[55]提出了一种新的图表示学习方法,用于在软件函数级别自动检测漏洞。他们将源代码表示为简化的代码属性图 SCPG (Simplified Code Property Graph), SCPG 是一种仅由 AST 和 CFG 组成的代码属性图 CPG。因此,可以尽可能多地保留源代码信息,同时确保图的规模足够小,以便高效处理。SCPG 的具体表示形式如图 3 所示,其每个节点包含 2 个部分:代码和类型。code 表示节点代表的源代码标记,type 表示 Joern^[56]中的 CPG 模式定义的节点类型。该方法在 *F1-score* 和召回率上分别达到 83.6%和 83.1%,高于现有的 CNN、RNN 方法;它还实现了 89.2%的最佳准确率和 84.0%的可接受精度。尽管如此,该方法还有一些局限性:首先,该文在实验中使用合成数据集而不是真实数据集;其次,所提出的方法是二进制分类,只能检测函数的源代码是否易受攻击,如何检测多种不同类型的漏洞仍然是一个悬而未决的问题。

现有的基于图神经网络的源代码漏洞检测方法构造的源代码表示图,其节点之间的连接大多数都是单向的,这将导致上下文信息被部分屏蔽。Cao 等人^[57]设计了 BGNN4VD 来聚合来自不同图的不同边的信息并区分前向信息和后向信息。通过引入前向边及后向边,可以加快信息传播速度并提高漏洞检测的准确性。这是一种通过构建双向图神经网络 BGNN (Bidirectional Graph Neural Network)的漏洞检测方法,该方法在 Step 2 阶段结合 AST、CFG 和 DFG,提出了代码复合图 CCG,用于捕获源代码的各种语法和语义信息;在 Step 3

阶段使用矢量化源代码作为 BGNN 的输入,BGNN 可以向前和向后传播用于捕获源代码之间的相互依赖关系的节点信息,包括代码表示、节点标签及其与其他邻居相关的链接信息,适用于非顺序输出和语义学习;在 Step 4 阶段使用 CNN 进一步提取特征并通过分类器检测漏洞。实验评估结果表明,相比现有最先进的方法,BGNN4VD 具有更高的精度和准确度。但是,该方法仍然有进一步改进的空间,例如,可以尝试开发针对不同编程语言的通用图挖掘工具,以提取不同类型的代码图。

漏洞触发的代码通常涉及一个或多个函数过程,而目前的研究更多地关注单个过程上的代码。由于缺乏对多个漏洞相关过程的综合分析,目前的方法在检测过程间漏洞方面存在误报和漏报。Song 等人^[58]针对此问题提出了 HGIVul (detecting Inter-procedural Vulnerabilities based on HyperGraph convolution),它是一种基于超图卷积的过程间漏洞检测方法。在 Step 1 阶段,Song 等人^[59]基于 D2A 构建了一个用于过程间分析的数据集,这是他们所知道的唯一一个从真实项目中构建的用于过程间分析的数据集,然后通过静态分析器 Infer 定位跨多个过程的潜在漏洞相关代码轨迹;在 Step 2 阶段,HGIVul 从跟踪中重建过程间控制流图 ICFG (Inter-procedural Control Flow Graph),以恢复多过程代码之间的复杂关系;在 Step 3 阶段,HGIVul 在 ICFG 上执行多级图卷积,以掌握多个过程中的整体代码特征;最后在 Step 4 阶段,将分类器应用于提取的代码特征以进行漏洞检测。实验结果显示,HGIVul 优于 Vul-DeePecker^[60]、DeepWukong^[61]、SySeVR^[62]、CBERT^[63]这几种现有的最先进的方法,并且在开源项目中提供了更好的检测结果。该方法还可以从多方面进行改进,如进一步增强漏洞检测性能、扩展多个漏洞的识别、探索更好的代码表示方法等。

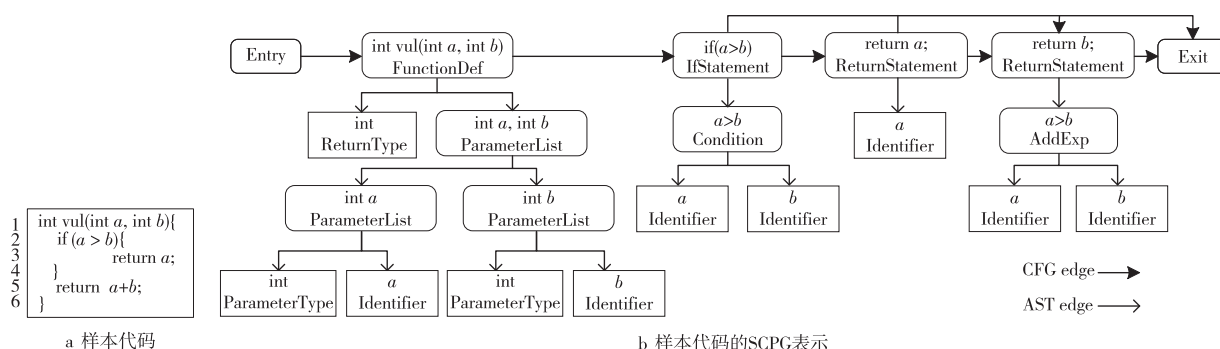


Figure 3 SCPG representation of the sample code

图3 样本代码的 SCPG 表示

基于函数级别的漏洞检测粒度以函数为单位进行漏洞检测,可以将每个函数视为独立的实体进行分析,减少了整个代码的复杂度。通过对函数进行分析,可以捕获函数之间的调用关系、参数传递等信息,提供了较为全面的代码信息覆盖量。

3.4 切片级别

基于切片级别的检测技术在 GNNVulD 中的 Step 1 收集和整理源代码数据集阶段,包括包含多个切片(语句)的软件项目,确保数据集中包含正常代码和已知漏洞的代码示例。在 Step 2 构建代码图阶段,将每个切片(语句)表示为图结构,其中每个节点代表一行代码语句,边代表切片中代码语句之间的控制流关系和数据依赖关系。构建代码图可以根据 AST、CFG 和 DFG 等技术进行。

现有大多数工作都是在函数级别或以更高的粒度检测漏洞代码的存在,因此,开发人员仍需耗费大量时间来检查代码以找到具体需要修复的漏洞代码语句,导致了不必要的工作量和时间浪费。

Zheng 等人^[64]提出了一种新的切片级漏洞检测方法 VulSPG(Vulnerability detection based on Slice Property Graph)。VulSPG 在 Step 2 阶段的代码图表示上,首先使用开源工具 Joern 对源代码进行解析,生成一系列代码表示(例如 AST、PDG、CPG);然后使用 SyVC(Semantic-based Vulnerability Candidates)^[62]作为一组切片的标准(该标准以 API/库函数调用、数组使用、指针使用、算术表达式、函数参数和函数返回语句为切点对代码属性图 CPG 进行切片),定义了一种名为切片属性图 SPG(Slice Property Graph)的新型代码表示。SPG 的不同之处在于,它的目的不仅是保留可能与漏洞相关的丰富语义和显式结构信息,而且还尽可能消除不相关信息,以降低漏洞的复杂性。在 Step 3 阶段使用改进的关系图卷积网络 R-GCNs(Relational Graph Convolutional Networks)模型和三重注意力机制来识别 SPG 中的潜在漏洞,其优点是可以高效、准确地捕获 SPG 中与漏洞相关的特征,用于漏洞检测任务。同时 Zhang 等人^[64]讨论了 VulSPG 尚存的一些局限性,如 VulSPG 目前专注于检测 C/C++ 程序源代码中的漏洞,所使用的切片标准对于一些漏洞代码的覆盖还是不够全面,在模型比较方面仅考虑了最常用的漏洞检测方法 GGNN 作为图编码的比较。

基于切片级别检测技术的关键就是对数据进行切片,不同的切片标准对模型的影响不同,因此选择一个符合漏洞检测模型的切片标准是很重要的。

在切片标准上宋子韬等人^[65]选择 VulDeePecker 和 SySeVR 中使用的 4 种类型的切点(系统 API 调用、数组使用、指针使用、算术操作)^[66]并在此基础上添加了返回语句、条件语句作为感兴趣的切点类型,增加了切片对源代码的覆盖。宋子韬等人^[65]使用 Joern 工具将源代码转换为程序依赖图 PDG。在 Step 2 阶段通过切片技术对 PDG 进行切片得到切片依赖图 SDG(Slice Dependency Graph);然后使用 Doc2Vec^[67]模型生成切片依赖图中每个节点的初始向量。在 Step 3 阶段使用 PyTorch Geometric^[54]框架实现基于线性特征调制的图神经网络 GNN-FiLM(Graph Neural Networks with Feature-wise Linear Modulation)^[68]来学习 SDG 中的信息,GNN-FiLM 网络的优势是可以更好地利用 SDG 中边的类型信息,更适用于学习有向图的特征,且性能优于 GAT 等网络模型。由于数据集的限制,宋子韬等人^[65]提出的方法的性能可能受到数据的影响,研究构建更高质量的漏洞数据集用于模型训练将会是一项挑战。

Cheng 等人^[61]介绍了一种基于深度学习的新型嵌入方法 DeepWukong,用于静态检测 C/C++ 程序的软件漏洞。在 Step 2 阶段 DeepWukong 首先构造基于控制流和数据流信息的程序依赖图 PDG,PDG 是一个有向图,其中每个节点代表一条指令(程序语句),每条边代表 2 个语句之间的数据或控制依赖关系。其次通过从程序切片基准点^[69](程序切片基准点通常来自 API 调用或用户指定的基准点)开始沿着 PDG 进行前向和反向遍历,直到固定点为程序生成每个切片。最后使用 Doc2Vec^[67]将源代码的每个语句的标记转换为向量表示。在 Step 3 阶段使用 Cangea 等人^[70]提出的图神经架构(包括图卷积层、图池化层和图读出层)来执行切片图的分类任务。DeepWukong 支持更精确的漏洞预测,以更细粒度的程序切片级别定位漏洞。DeepWukong 还进行了更精确的过程间分析,这对于现实世界的程序至关重要,因为函数调用在现代软件项目中非常常见。DeepWukong 在图嵌入中保留了全面的代码特征(即过程间控制和数据流),能够捕获漏洞的共同特征,并极大地提高了现有基于深度学习的方法的性能。尽管如此,DeepWukong 方法仍存在一些不完美的地方,如:数据集中某些样本的漏洞标签可能被贴错,代码切片选取的切片兴趣点可能会错过一些极端的情况,该方法的实验仅限于 C/C++ 程序中的漏洞。

切片级别的源代码漏洞检测存在的一大难点就是切片标准的制定,因此,在切片标准上直接选择语句粒度的切片也不失为一种简单有效的方法。Ding 等人^[71]提出了 VELVET (noVel Ensemble Learning approach to automatically locate Vulnerable Statements) 框架,这是一种用于定位漏洞语句的新型集成学习方法。VELVET 使用 Joern 在 Step 2 阶段生成代码属性图 CPG 来表示程序的图语义(图中每个节点表示一条代码语句),使用 word2vec 模型对节点信息向量化。在 Step 3 阶段使用结合了基于图和基于序列的神经网络模型(GGNN^[33]和 Transformer^[72]),来捕获程序图的局部和全局上下文信息,保留程序代码语义和漏洞模式信息。Ding 等人^[71]使用了合成的数据集在单个语句级别进行漏洞检测,他们设计的对合成数据进行预训练然后对真实世界数据进行微调的工作流程为现实世界数据集稀缺问题提供了实用的解决方案。但是,该方法建立在类似漏洞在过去发生过的前提下。

Hin 等人^[73]提出了一种新的语句级别软件漏洞检测框架,即 LineVD。LineVD 是第 1 种通过图神经网络从函数级和语句级信息中联合学习以增强软件漏洞检测性能的方法,在实证评估中它显著优于仅使用一种类型信息的传统模型。它将语句级漏洞检测制定为节点分类任务。LineVD 在 Step 3 阶段引入图注意力网络 GAT 来构建从图中学习拓扑依赖信息的模型,然后使用 CodeBERT 模型^[74]进行特征提取,最后针对从多个真实世界项目中获得的大量真实世界 C/C++ 漏洞进行了广泛的实验,并证明其 *F1-score* 相比当前最先进的技术水平提高了 105%。虽然 LineVD 优于当前最先进的技术,但其性能仍然很低,这意味着语句级的漏洞检测任务仍有很大的进步空间。Hin 等人^[73]指出使用针对特定目标语言预训练的大型语言模型(例如 C-BERT)可以提高下游任务的性能。

Cao 等人^[75]提出了 MVD (Memory-related Vulnerability Detection based on flow-sensitive graph neural networks),一种基于流敏感图神经网络 FS-GNN (Flow-Sensitive Graph Neural Networks) 的语句级内存相关漏洞检测方法。FS-GNN 用于联合嵌入非结构化信息(即源代码)和结构化信息(即控制流和数据流),以捕获与隐式内存相关的漏洞模式。在 Step 2 阶段,MVD 基于程序的控制流和数据流使用静态分析工具 Joern 来解

析源代码并构建程序依赖图 PDG,并且使用调用图 CG(Call Graph)获得的函数调用关系和返回值扩展 PDG 以进行过程间分析,从而捕获全面和精确的程序语义。为了减少不相关的语义噪声,MVD 从程序兴趣点进行程序切片^[70,76],然后使用 Doc2Vec^[67]将每个切片的语句转换为低维向量表示。在 Step 3 阶段 MVD 使用 FS-GNN 联合嵌入节点和关系学习隐式漏洞模式并重新平衡节点标签分布。FS-GNN 的关键是联合嵌入语句和嵌入流信息,以捕获用于语义学习的敏感上下文信息。实验结果表明,MVD 在准确性和效率之间做出了很好的权衡。MVD 在实验中仍存在一定局限性,其实验仅限于 C/C++ 程序中与内存相关的漏洞,当应用于更复杂的漏洞或语言(例如 Java)时,这些实验的可重现性可能无法保证。但是,该方法是通用的,可以扩展到其他漏洞和语言。

切片级别的漏洞检测相对于文件级别和函数级别具有一些独特的优点。切片级别的漏洞检测通过将源代码切割成更小的片段,可以实现更精细的粒度和准确的漏洞定位;它能够更准确地捕获与漏洞相关的特征和模式,从而提高检测的精度。此外,切片级别的漏洞检测还能够提供更具体和详细的漏洞报告,有助于开发人员理解和修复漏洞。但是,切片级别的漏洞检测也面临一些挑战,包括高计算复杂度和数据稀疏性的问题。因此,在实际应用中,需要综合考虑不同粒度的优缺点,并根据具体的场景和需求选择合适的漏洞检测粒度。

3.5 小结

从文件级别、函数级别和切片级别 3 个角度对基于图神经网络的源代码漏洞检测技术进行讨论的原因在于这 3 种粒度代表了漏洞检测的不同层次和视角,有助于全面分析和评估基于 GNN 的方法在不同情境下的表现和适用性。这有助于研究人员和开发人员选择合适的粒度来应对不同类型的漏洞和问题,以及更好地平衡准确性和效率。

本文在表 2 对基于 GNN 的源代码漏洞检测模型进行了总结。不同数据集来源、不同的代码表示、不同的 GNN 模型以及不同的漏洞检测粒度对基于图神经网络的源代码漏洞检测的效果都会有一定的影响。

(1)数据集来源有开源的数据集和自定义数据集。①开源数据集:开源数据集通常具有更广泛的代码覆盖范围和多样性,可以更好地代表真实世界的代码。使用开源数据集进行训练和评估可以更

Table 2 Vulnerability detection model of source code based on GNN
表 2 基于 GNN 的源代码漏洞检测模型

模型	语言	数据集来源	检测粒度	代码表示	GNN 模型
DeepTective ^[44]	PHP	SARD,GitHub	函数级别+文件级别	CFG	GCN+GRU
文献[48]模型	Java	OWASP	文件级别	AST/CFG/PDG	GCN+GAT
BGNN4VD ^[57]	C/C++	NVD,Github	函数级别	AST+CFG+DFG=CCG	BGNN
FUNDED ^[50]	C/Java/Swift/PHP	CVE,SARD	函数级别	AST+CDFG	GGNN
LLVM IR ^[51]	C,Fortran 等	SARD	函数级别	CFG	GCN
HGIVul ^[58]	C/C++	D2A	函数级别	ICFG	GCN
VELVET ^[71]	C/C++	JULIET,D2A	切片级别	AST+CFG+DFG=CPG	GGNN+Transformer
DeepWukong ^[61]	C/C++	SARD	切片级别	CDG+VFG=PDG	GCN+GAT+k-GNNs
LineVD ^[73]	C/C++	CVE,Github	函数级别+语句级别	PDG	GAT
VulSPG ^[64]	C/C++	NVD,SARD	切片级别	DDG+CDG+FCDG=SPG	R-GCN+triple attention
MVD ^[75]	C/C++	SARD,CVE	语句级别	DFG+CFG+CG=PDG	FS-GNN
文献[65]模型	C/C++	CWE,SARD	切片级别	SDG	GNN-FiLM

好地推广和比较不同的方法。②自定义数据集:自定义数据集更专注于特定的漏洞类型或特定领域的代码。它们可以提供更精确的漏洞标注和更贴近实际应用的场景,但可能缺乏大规模的数据量。

(2)代码表示为不同的图结构所能覆盖的源代码信息不同,图越复杂覆盖信息越多,但图神经网络的计算难度也呈指数增加。因此,要基于自己的研究需要对源代码进行构图表示。

(3)不同的 GNN 模型在处理图数据和学习特征方面具有不同的能力和表达能力。一些模型如 GCN 适用于节点级别的特征学习,而图注意力网络则能够更好地处理节点之间的关系和权重。选择适合特定任务和数据集的 GNN 模型可以提高漏洞检测的性能。

(4)文件级别、函数级别和切片级别的检测粒度涵盖了不同层次的代码结构。文件级别检测可以更全局地捕捉漏洞,函数级别检测更关注函数内的漏洞,而切片级别检测则更注重局部的代码片段。不同粒度的检测可以提供不同级别的漏洞发现和定位能力。

对于这些影响因素,需要考虑具体的研究问题和任务需求。合理选择数据集来源、代码表示、GNN 模型和检测粒度,以及进行实验评估和对比分析,可以提供对源代码漏洞检测效果的更全面和准确的理解,并帮助研究人员选择适合特定场景的方法和技术。

基于图神经网络的源代码漏洞检测在不同粒度上有各自的优缺点。文件级别检测具有全局视角,能综合分析整个源代码文件,捕捉文件级漏洞

模式,但可能无法精确定位具体漏洞位置。函数级别检测精确度高,能定位具体漏洞并提供修复建议,但可能忽略文件级的整体信息。切片级别检测基于代码执行路径分析,捕捉细粒度漏洞特征,但需更多计算资源。在实际应用中可根据需求选择合适的粒度,权衡准确性和资源消耗。

本文针对 3 种不同的漏洞检测粒度进行了横向对比分析,如表 3 所示。

Table 3 Side-by-side comparison
of different detection particle sizes
表 3 不同检测粒度的横向对比

检测粒度	优点	缺点
文件级别	全局视角	上下文信息丢失
	快速筛查	模型复杂
函数级别	准确定位漏洞	忽略全局信息
	上下文信息保留	数据流分析复杂
切片级别	精确漏洞定位	切片选择困难
	重点关注漏洞区域	全局信息丢失

(1)文件级别:文件级别的漏洞检测是在整个源代码文件上进行的检测。它关注的是源代码文件中是否存在潜在的漏洞,而不考虑具体的函数或代码片段。文件级别的漏洞检测通常用于快速筛查和初步评估,可以帮助开发人员在整体上了解代码的安全性。因为无法精确定位到具体的函数或代码片段,并且在处理大规模文件的图结构时可能导致模型计算和内存要求较高,因此,文件级别的漏洞检测可能会丢失某些上下文信息。

(2)函数级别:函数级别的漏洞检测是在函数

的级别上进行检测。它关注的是每个函数内部是否存在漏洞,以及函数之间的调用关系。函数级别的漏洞检测更加细粒度,可以更准确地定位漏洞,并提供更详细的信息和修复建议。但是,函数级别的漏洞检测可能忽略了函数之间的全局依赖和全局结构,无法全面分析整个代码的安全性;并且函数级别的数据建模需要进行复杂的数据流分析,需要解决数据流路径的问题。

(3)切片级别:切片级别的漏洞检测是在代码切片的级别上进行检测。切片是指从源代码中提取出与某个特定语句相关的代码片段。切片级别的漏洞检测关注的是与特定漏洞相关的代码切片,以便更精确地分析和修复漏洞。但是,切片级别的建模需要准确地选择和提取与特定语句相关的代码切片,这可能涉及到切片选择的难题;并且因为只关注特定语句周围的代码,切片级别的建模可能丢失一些全局的漏洞信息。

在基于图神经网络的源代码漏洞检测任务中,横向上按漏洞检测粒度分为文件级别、函数级别和切片级别,而在纵向上3种检测粒度可以看作是从全局视野到局部视野的递进关系,各有特点,应用

场景也各不相同。表4对其纵向关系进行了直观对比。

这3种漏洞检测粒度的递进关系可以更细致地分析和定位源代码中的漏洞问题。从文件级别到函数级别再到切片级别,检测的范围逐渐缩小,分析的粒度逐渐细化,从宏观到微观逐步深入源代码的结构和依赖关系。在研究工作中,可以根据具体的需求和任务,选择适当的漏洞检测粒度来进行源代码的安全分析。

4 漏洞数据集

为了便于对比,本文对第3节中所使用的数据集进行了列表对比展示。对于每一篇文献所提模型,详细列举了7项统计信息,包括数据集样本粒度、数据集涉及的编程语言、拥有的漏洞类型数(其中,“-”表示未区分漏洞类型)、数据集总样本数、漏洞样本数、漏洞样本占比和数据集是否开源。

统计结果如表5所示。从表5中可以看出,当前漏洞代码数据集在各项技术环节存在较大差异,如语言和粒度等;大多数数据集存在正负样本不均

Table 4 Longitudinal comparison of different detection particle sizes

表 4 不同检测粒度的纵向对比

检测粒度	检测范围	特点	应用场景
文件级别	整个源代码文件	关注整个文件中是否存在潜在的漏洞问题	适用于快速筛查和整体评估,对文件级别的安全性进行初步评估
函数级别	单个函数	关注每个函数内部是否存在漏洞,如函数调用关系、数据依赖等	适用于更详细的漏洞定位和修复建议,提供函数级别的安全性分析
切片级别	特定语句相关的代码切片	关注特定语句周围的代码片段,具体到与语句相关的漏洞分析	关注特定语句周围的代码片段,具体到与语句相关的漏洞分析

Table 5 Summary of vulnerability dataset information

表 5 漏洞数据集信息总结

模型	样本粒度	语言	漏洞类型数/种	总样本数/个	漏洞样本数/个	漏洞样本比例/%	是否开源
DeepTective ^[44]	文件级别	PHP	3	5 454	2 728	50.01	是
文献[48]模型		Java	7	1 698	902	53.12	否
BGNN4VD ^[57]	函数级别	C/C++	—	95 925	3 867	4.03	是
FUNDED ^[50]		C/Java/Swift/PHP	64	150 950	75 475	50.00	否
LLVM IR ^[51]		C、Fortran 等	3	48 550	24 275	50.00	否
HGIVul ^[58]		C/C++	10	566 968	12 290	2.16	否
VELVET ^[71]	切片级别	C/C++	9	5 000	2 500	50.00	否
DeepWukong ^[61]		C/C++	10	569 628	158 118	27.76	是
LineVD ^[73]		C/C++	91	187 000	10 000	5.35	是
VulSPG ^[64]		C/C++	—	264 532	84 652	32.00	否
MVD ^[75]		C/C++	13	8 706	4 353	50.00	是
文献[65]模型		C/C++	5	220 056	61 826	28.10	否

衡问题;超过半数的数据集未开源,给其他研究人员进行模型复现和横向对比带来困难。

从样本粒度来看,不同粒度的样本在完整度和冗余度上有差距。文件级数据包含较多路径,但会带来大量的无关噪声。函数级别数据划分成本低,检测过程中无需分析被测代码(仅需切割为函数)。然而,函数级数据基于的朴素假设是漏洞发生在函数范围内,在大规模工业代码中,特别是模块化情况下,数据声明与使用往往不在单一函数内,这一假设难以成立。切片级别数据支持跨函数漏洞路径,但切片操作需在构建数据集和检测被测代码时进行数据流、控制流等分析,资源需求大、时间成本高。

为了克服这些问题,未来的研究应该致力于建立更大规模的数据集,涵盖不同语言、粒度和漏洞类型,以解决目前数据集的不均衡问题;同时,提高数据集的开源性,促进模型复现和横向对比研究。

目前基于图神经网络的源代码漏洞检测方法通常难以直接处理零样本学习 ZSL (Zero-Shot Learning) 问题。ZSL 问题指的是在模型训练阶段没有见过某些特定类型的漏洞样本,但在实际应用中需要对这些未见过的漏洞进行检测。构建数据集是解决 ZSL 问题的一种常见方法。通过收集和标注更多类型的漏洞样本,可以增加数据集的多样性,并覆盖更广泛的漏洞类型。这样,模型在训练阶段可以接触到更多种类的漏洞,提高对未见过漏洞的泛化能力。

而数据集的构建一直是基于图神经网络的源代码漏洞检测任务的难点之一。目前的研究发现大语言模型可以在一定程度上弱化数据集质量和多样性的要求,进一步提高源代码漏洞检测的效果和性能。

大语言模型,如 GPT (Generative Pre-trained Transformer), 可以通过预训练和微调的方式,为源代码漏洞检测提供一种基于生成的方法。具体而言,可以将大语言模型训练于大规模的源代码数据集上,使其学习源代码的语法和语义特征;然后,通过对预训练模型与特定的漏洞检测任务进行微调,将模型的知识迁移至新的漏洞类型上,从而实现 ZSL 漏洞检测。

使用大语言模型的优势在于它们能够生成具有语法和语义一致的源代码,甚至可以生成具有漏洞特征的代码片段。这样,即使数据集质量和多样性有限,大语言模型也可以通过生成漏洞代码样本进行模型训练和评估。

然而,还需要注意以下几点:

(1) 数据集的多样性: 尽管大语言模型可以生成漏洞代码样本,但数据集的多样性仍然很重要。多样性的数据集有助于模型更好地理解源代码的特征和漏洞模式,提高模型的泛化能力。

(2) 漏洞样本的标注: 为了训练和评估模型,仍然需要一定数量的已标注漏洞样本。这需要专业的安全专家对漏洞进行准确标注,以确保训练出的模型能够有效地检测漏洞。

(3) 模型的解释性: 大语言模型通常是黑盒模型,其生成的漏洞代码可能难以解释。为了满足审计和验证的要求,需要进一步研究如何解释和理解大语言模型在源代码漏洞检测中的预测结果。

因此,在使用大语言模型进行源代码漏洞检测时,仍需要综合考虑数据集的质量、多样性以及模型的可解释性,以充分发挥大语言模型的优势,并在实践中不断探索改进方法。

5 现实挑战和未来展望

基于图神经网络的源代码漏洞检测技术已经取得了一定的进展,不断有新的方法被提出并且逐渐得到应用。随着技术的不断发展和完善,这些方法将能够更加准确地检测出源代码中的各种漏洞,并且具有更强的泛化能力。但是,基于图神经网络的方法也有一些挑战需要克服。

(1) 现实挑战。

虽然现有的基于图神经网络的源代码漏洞检测研究已经解决了一些存在的问题,但该领域还处于初步研究阶段,在实践中仍面临着一些挑战:

① 数据表示: 将源代码表示为图是关键的一步,但如何准确地表示源代码中的结构和依赖关系仍然是一个挑战。代码的复杂性和多样性使得构建准确且具有可解释性的图表示变得困难。

② 数据规模: 源代码通常非常庞大,包含大量的文件和函数。构建和处理大规模图数据需要有效的计算和存储策略,以及高效的图神经网络模型和算法。

③ 标注数据: 源代码漏洞的标注数据相对稀缺。由于漏洞的多样性和复杂性,手动标注大规模源代码是一项耗时且困难的任务。缺乏足够的标注数据限制了图神经网络模型的训练和性能评估。

④ 噪声和冗余信息: 源代码中可能存在大量的冗余和噪声信息,例如注释、格式化等。这些信息

可能会对图神经网络的性能产生负面影响,需要有效的数据清洗和预处理方法。

⑤可解释性和可视化:图神经网络模型通常被认为是黑盒模型,缺乏对结果的解释和可视化能力。在源代码漏洞检测中,理解模型如何推断和预测漏洞是至关重要的,因此提高模型的可解释性和可视化能力是一个挑战。

⑥跨编程语言和代码库:不同的编程语言和代码库具有不同的代码结构和规范。跨编程语言和代码库的源代码漏洞检测需要模型具备一定的通用性和泛化能力,以适应不同的环境和语境。

解决这些挑战需要进一步的研究和创新。通过改进数据表示方法、开发更高效的图神经网络模型和算法、积累更多的标注数据,并探索模型的可解释性和可视化方法,可以提高基于图神经网络的源代码漏洞检测的实际应用能力。

(2)未来展望。

未来的研究方向将着重解决基于图神经网络的源代码漏洞检测面临的现实挑战:

①数据表示和特征提取方法的改进:研究人员可以探索更有效的方法来准确地表示源代码的图结构和依赖关系,以及提取更有用的特征。这可能涉及到结合语义分析、程序语言理解和代码规范等方面的技术。

②大规模图数据处理和模型优化:针对源代码的大规模图数据,研究人员可以开发更高效的图处理算法和模型优化方法。这包括针对图数据的存储、计算和训练的策略,以提高模型的效率和可扩展性。

③弱监督和半监督学习:考虑到源代码漏洞标注数据的稀缺性,研究人员可以探索弱监督和半监督学习方法,通过利用部分标注数据和领域知识来增强模型的训练能力。

④跨编程语言和代码库的通用性:为了更好地适应不同编程语言和代码库的源代码漏洞检测需求,研究人员可以尝试开发通用的图神经网络模型和算法。这可能涉及到跨编程语言的特征转换和语言无关的漏洞检测方法。

⑤可解释性和可视化方法的研究:为了提高图神经网络模型在源代码漏洞检测中的可解释性和可视化能力,研究人员可以探索新的方法和技术,使模型的决策和预测结果更易于理解和解释。

⑥结合其他技术手段的综合方法:图神经网络可以与其他技术手段结合,如程序分析、符号执行和静态分析等。研究人员可以探索如何将这些技

术与图神经网络相结合,以提高漏洞检测的准确性和全面性。

综上所述,未来的研究方向包括改进数据表示和特征提取方法、处理大规模图数据、利用弱监督和半监督学习、实现跨编程语言和代码库的通用性、提升可解释性和可视化能力,以及结合其他技术手段等。通过这些方向上的持续努力,基于图神经网络的源代码漏洞检测技术有望在未来得到进一步的发展和突破,提高漏洞检测的准确性、效率和可靠性,从而更好地保护软件系统的安全性和稳定性。

6 结束语

将源代码表示为图结构并利用图神经网络进行漏洞检测,可以充分挖掘代码结构信息,提取丰富的特征,从而提高检测准确性和效果。基于图神经网络的源代码漏洞检测技术的研究为解决开源安全问题提供了强有力的解决方案,推动了源代码漏洞检测领域的创新,为今后更安全的开源软件开发提供了强有力的支持和保障。因此,基于图神经网络的源代码漏洞检测技术的研究对解决开源安全问题具有深远的意义。基于以上内容,本文提出了一个基于图神经网络的源代码漏洞检测通用框架 GNNVulD,以此框架为基础、基于漏洞检测粒度,从文件级别、函数级别和切片级别 3 个方面系统地归纳和阐述了现有方法以及相关数据集,并提出漏洞数据集存在的相关问题以及一些解决措施。最后讨论了该领域所面临的挑战,并对未来可能的研究重点进行了展望。

参考文献:

- [1] 纪守领,王琴应,陈安莹,等. 开源软件供应链安全研究综述[J]. 软件学报,2023,34(3):1330-1364.
Ji Shou-ling, Wang Qin-ying, Chen An-ying, et al. Survey on open-source software supply chain security[J]. Journal of Software, 2023, 34(3): 1330-1364.
- [2] Millar S. Vulnerability detection in open source software: An introduction[J]. arXiv:2203.16428, 2022.
- [3] Söderberg E, Church L, Höst M. Open data-driven usability improvements of static code analysis and its challenges[C]// Proc of the 25th International Conference on Evaluation and Assessment in Software Engineering, 2021: 272-277.
- [4] State of software security: Open source edition [EB/OL]. [2023-12-27]. <https://www.veracode.com/sites/default/files/pdf/resources/reports/state-of-software-security-open-source-edition-veracode-report.pdf>.

- [5] Data center: AT TOKYO's data centers[EB/OL]. [2023-12-27]. <https://www.attokyo.com/datacenter/>.
- [6] Li Z, Zou D Q, Xu S H, et al. VulDeeLocator: A deep learning-based fine-grained vulnerability detector [J]. IEEE Transactions on Dependable and Secure Computing, 2022, 19(4): 2821-2837.
- [7] Zou D Q, Wang S J, Xu S H, et al. μ VulDeePecker: A deep learning-based system for multiclass vulnerability detection [J]. IEEE Transactions on Dependable and Secure Computing, 2021, 18(5): 2224-2236.
- [8] Hanif H, Maffei S. VulBERTa: Simplified source code pre-training for vulnerability detection[C]//Proc of 2022 International Joint Conference on Neural Networks, 2022: 1-8.
- [9] Wu B L, Zou F T. Code vulnerability detection based on deep sequence and graph models: A survey[J]. Security and Communication Networks, 2022, 2022: 1176898.
- [10] Lin W, Cai S H. An empirical study on vulnerability detection for source code software based on deep learning[C]//Proc of 2021 IEEE 21st International Conference on Software Quality, Reliability and Security Companion, 2021: 1159-1160.
- [11] Wu Z H, Pan S R, Chen F W, et al. A comprehensive survey on graph neural networks[J]. IEEE Transactions on Neural Networks and Learning Systems, 2021, 32(1): 4-24.
- [12] Gao H, Cheng S Y, Zhang W M. GDroid: Android malware detection and classification with graph convolutional network[J]. Computers & Security, 2021, 106: 102264.
- [13] Zhou J, Cui G Q, Hu S D, et al. Graph neural networks: A review of methods and applications[J]. AI Open, 2020, 1: 57-81.
- [14] Chakraborty S, Krishna R, Ding Y, et al. Deep learning based vulnerability detection: Are we there yet? [J]. IEEE Transactions on Software Engineering, 2022, 48(9): 3280-3296.
- [15] Nguyen H V, Zheng J J, Inomata A, et al. Code aggregate graph: Effective representation for graph neural networks to detect vulnerable code[J]. IEEE Access, 2022, 10: 123786-123800.
- [16] Katsadourous E, Patrikakis C. A survey on vulnerability prediction using GNNs[C]//Proc of the 26th Pan-Hellenic Conference on Informatics, 2022: 38-43.
- [17] Fan Y H, Wan C H, Fu C, et al. VDoTR: Vulnerability detection based on tensor representation of comprehensive code graphs[J]. Computers & Security, 2023, 130: 103247.
- [18] Chernis B, Verma R. Machine learning methods for software vulnerability detection[C]//Proc of the 4th ACM International Workshop on Security and Privacy Analytics, 2018: 31-39.
- [19] Gong J, Kuang X H, Liu Q. Survey on software vulnerability analysis method based on machine learning [C]//Proc of 2016 IEEE 1st International Conference on Data Science in Cyberspace, 2016: 642-647.
- [20] Jia Y T, Zhuang H L, Lin Z C, et al. Machine learning for software vulnerability analysis: A survey[C]//Proc of 2021 IEEE 6th International Conference on Data Science in Cyberspace, 2021: 396-402.
- [21] 李韵, 黄辰林, 王中锋, 等. 基于机器学习的软件漏洞挖掘方法综述[J]. 软件学报, 2020, 31(7): 2040-2061.
- Li Yun, Huang Chen-lin, Wang Zhong-feng, et al. Survey of software vulnerability mining methods based on machine learning[J]. Journal of Software, 2020, 31(7): 2040-2061.
- [22] Wang J J, Huang M H, Nie Y P, et al. Static analysis of source code vulnerability using machine learning techniques: A survey[C]//Proc of 2021 4th International Conference on Artificial Intelligence and Big Data, 2021: 76-86.
- [23] Zeng P, Lin G J, Pan L, et al. Software vulnerability analysis and discovery using deep learning techniques: A survey[J]. IEEE Access, 2020, 8: 197158-197172.
- [24] Zhang Y L. Support vector machine classification algorithm and its application[M]. Information Computing and Applications. Berlin: Springer, 2012: 179-186.
- [25] Breiman L. Random forests[J]. Machine Learning, 2001, 45(1): 5-32.
- [26] O'Shea K, Nash R. An introduction to convolutional neural networks[J]. arXiv: 1511. 08458, 2015.
- [27] Lipton Z C, Berkowitz J, Elkan C. A critical review of recurrent neural networks for sequence learning[J]. arXiv: 1506. 00019, 2015.
- [28] Kipf T N, Welling M. Semi-supervised classification with graph convolutional networks[J]. arXiv: 1609. 02907, 2017.
- [29] Veličković P, Cucurull G, Casanova A, et al. Graph attention networks[J]. arXiv: 1710. 10903, 2018.
- [30] Gori M, Monfardini G, Scarselli F. A new model for learning in graph domains[C]//Proc of 2005 IEEE International Joint Conference on Neural Networks, 2005: 729-734.
- [31] Scarselli F, Gori M, Tsoi A C, et al. The graph neural network model[J]. IEEE Transactions on Neural Networks, 2009, 20(1): 61-80.
- [32] Hamilton W L, Ying R, Leskovec J. Inductive representation learning on large graphs[C]//Proc of the 31st International Conference on Neural Information Processing Systems, 2017: 1025-1035.
- [33] Li Y J, Tarlow D, Brockschmidt M, et al. Gated graph sequence neural networks[J]. arXiv: 1511. 05493, 2015.
- [34] Baxter I D, Yahin A, Moura L, et al. Clone detection using abstract syntax trees[C]//Proc of International Conference on Software Maintenance, 1998: 368-377.
- [35] Yamaguchi F, Lottmann M, Rieck K. Generalized vulnerability extrapolation using abstract syntax trees[C]//Proc of the 28th Annual Computer Security Applications Conference, 2012: 359-368.
- [36] Allamanis M, Brockschmidt M, Khademi M. Learning to represent programs with graphs[J]. arXiv: 1711. 00740, 2017.
- [37] Alon U, Brody S, Levy O, et al. Code2seq: Generating se-

- quences from structured representations of code[J]. arXiv: 1808.01400, 2018.
- [38] Mou L L, Li G, Zhang L, et al. Convolutional neural networks over tree structures for programming language processing[C]//Proc of the 30th AAAI Conference on Artificial Intelligence, 2016: 1287-1293.
- [39] Schrouff J, Wohlfahrt K, Marnette B, et al. Inferring java-script types using graph neural networks[M]. arXiv: 1905.06707, 2019.
- [40] Liang H L, Sun L, Wang M L, et al. Deep learning with customized abstract syntax tree for bug localization[J]. IEEE Access, 2019, 7: 116309-116320.
- [41] Sparks S, Embleton S, Cunningham R, et al. Automated vulnerability analysis: Leveraging control flow for evolutionary input crafting[C]//Proc of the 23rd Annual Computer Security Applications Conference, 2007: 477-486.
- [42] Gascon H, Yamaguchi F, Arp D, et al. Structural detection of android malware using embedded call graphs[C]//Proc of the 2013 ACM Workshop on Artificial Intelligence and Security, 2013: 45-54.
- [43] Ferrante J, Ottenstein K J, Warren J D. The program dependence graph and its use in optimization[J]. ACM Transactions on Programming Languages and Systems, 1987, 9(3): 319-349.
- [44] Rabheru R, Hanif H, Maffei S. A hybrid graph neural network approach for detecting PHP vulnerabilities[C]//Proc of 2022 IEEE Conference on Dependable and Secure Computing, 2022: 1-9.
- [45] Clarke J. SQL 注入攻击与防御[M]. 北京: 清华大学出版社, 2013.
- Clarke J. SQL injection attacks and defense[M]. Beijing: Tsinghua University Press, 2013.
- [46] CWE-79: Improper neutralization of input during web page generation ('cross-site scripting')[EB/OL]. [2023-12-27]. <https://cwe.mitre.org/data/definitions/79.html>.
- [47] OSCI - Open source contributor index[EB/OL]. [2023-12-27]. <https://opensourceindex.io>.
- [48] Ghaffarian S M, Shahriari H R. Neural software vulnerability analysis using rich intermediate graph representations of programs[J]. Information Sciences, 2021, 553: 189-207.
- [49] Schütze H, Manning C D, Raghavan P. Introduction to information retrieval [M]. Cambridge: Cambridge University Press, 2008.
- [50] Wang H T, Ye G X, Tang Z Y, et al. Combining graph-based learning with automated data collection for code vulnerability detection[J]. IEEE Transactions on Information Forensics and Security, 2021, 16: 1943-1958.
- [51] 陈皓, 易平. 基于图神经网络的代码漏洞检测方法[J]. 网络与信息安全学报, 2021, 7(3): 37-45.
- Chen Hao, Yi Ping. Code vulnerability detection method based on graph neural network [J]. Chinese Journal of Network and Information Security, 2021, 7(3): 37-45.
- [52] opt-LLVM optimizer — LLVM 17.0.0 git documentation [EB/OL]. [2023-12-28]. <https://vm.org/docs//commandGuide/opt.html>.
- [53] Mikolov T, Chen K, Corrado G, et al. Efficient estimation of word representations in vector space [J]. arXiv: 1301.3781v3, 2013.
- [54] PyG Documentation — pytorch_geometric documentation [EB/OL]. [2023-12-25]. <https://pytorch-geometric.readthedocs.io/en/latest>.
- [55] Wu Y L, Lu J T, Zhang Y Y, et al. Vulnerability detection in C/C++ source code with graph representation learning[C]//Proc of 2021 IEEE 11th Annual Computing and Communication Workshop and Conference, 2021: 1519-1524.
- [56] Joern-The bug hunter's workbench[EB/OL]. [2023-12-27]. <https://joern.io>.
- [57] Cao S C, Sun X B, Bo L L, et al. BGNN4VD: Constructing bidirectional graph neural-network for vulnerability detection[J]. Information and Software Technology, 2021, 136: 106576.
- [58] Song Z H, Wang J F, Yang K Y, et al. HGIVul: Detecting inter-procedural vulnerabilities based on hypergraph convolution[J]. Information and Software Technology, 2023, 160: 107219.
- [59] Zheng Y H, Pujar S, Lewis B, et al. D2A: A dataset built for AI-based vulnerability detection methods using differential analysis[C]//Proc of 2021 IEEE/ACM 43rd International Conference on Software Engineering, 2021: 111-120.
- [60] Li Z, Zou D Q, Xu S H, et al. VulDeePecker: A deep learning-based system for vulnerability detection[C]//Proc of 2018 Network and Distributed System Security Symposium, 2018: 1-15.
- [61] Cheng X, Wang H Y, Hua J Y, et al. DeepWukong: Statically detecting software vulnerabilities using deep graph neural network[J]. ACM Transactions on Software Engineering and Methodology, 2021, 30(3): Article No. 38.
- [62] Li Z, Zou D Q, Xu S H, et al. SySeVR: A framework for using deep learning to detect software vulnerabilities[J]. IEEE Transactions on Dependable and Secure Computing, 2022, 19(4): 2244-2258.
- [63] Buratti L, Pujar S, Bornea M, et al. Exploring software naturalness through neural language models[J]. arXiv: 2006.12641, 2020.
- [64] Zheng W N, Jiang Y, Su X H. Vu1SPG: Vulnerability detection based on slice property graph representation learning [C]//Proc of 2021 IEEE 32nd International Symposium on Software Reliability Engineering, 2021: 457-467.
- [65] 宋子韬, 胡勇. 基于图神经网络的源码漏洞检测方法研究[J]. 通信技术, 2022, 55(5): 640-645.
- Song Zi-tao, Hu Yong. Source code vulnerability detection method based on graph neural network [J]. Communication Technology, 2022, 55(5): 640-645.
- [66] Wang X D, Wang S, Feng P B, et al. PatchDB: A large-scale

- security patch dataset[C]//Proc of 2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks,2021;149-160.
- [67] Le Q, Mikolov T. Distributed representations of sentences and documents[C]//Proc of the 31st International Conference on Machine Learning,2014;1188-1196.
- [68] Brockschmidt M. GNN-FiLM: Graph neural networks with feature-wise linear modulation[C]//Proc of the 37th International Conference on Machine Learning,2020;1144-1152.
- [69] Weiser M. Program slicing[J]. IEEE Transactions on Software Engineering,1984,SE-10(4):352-357.
- [70] Cangea C, Veličković P, Jovanović N, et al. Towards sparse hierarchical graph classifiers[J]. arXiv:1811.01287,2018.
- [71] Ding Y, Suneja S, Zheng Y H, et al. VELVET: A novel ensemble learning approach to automatically locate vulnerable statements[C]//Proc of 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering,2022:959-970.
- [72] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[C]//Proc of the 31st International Conference on Neural Information Processing Systems,2017:6000-6010.
- [73] Hin D, Kan A, Chen H M, et al. LineVD: Statement-level vulnerability detection using graph neural networks[C]//Proc of the 19th International Conference on Mining Software Repositories,2022:596-607.
- [74] Feng Z Y, Guo D Y, Tang D Y, et al. CodeBERT: A pre-trained model for programming and natural languages[J]. arXiv:2002.08155v4,2020.
- [75] Cao S C, Sun X B, Bo L L, et al. MVD: Memory-related vulnerability detection based on flow-sensitive graph neural networks[C]//Proc of 2022 IEEE/ACM 44th International Conference on Software Engineering,2022:1456-1468.
- [76] Soremekun E, Kirschner L, Böhme M, et al. Locating faults with program slicing: An empirical analysis[J]. Empirical Software Engineering,2021,26(3):51.

作者简介:



陈子雄(2000 -),男,山西长治人,硕士生,研究方向为网络安全和图深度学习。

E-mail:15534417845@163.com

CHEN Zi-xiong, born in 2000, MS candidate, his research interests include network security and deep learning on graphs.



陈旭(1979 -),男,宁夏银川人,博士,高级工程师,CCF 会员(P5110M),研究方向为网络安全、图深度学习和大数据处理。

E-mail:chenxu@nmu.edu.cn

CHEN Xu, born in 1979, PhD, senior engineer, CCF member(P5110M), his research interests include cyber security, graph deep learning and big data processing.



景永俊(1977 -),男,宁夏银川人,硕士,高级工程师,CCF 会员(43390M),研究方向为图数据挖掘和异常检测。**E-mail:**jingyj@nmu.edu.cn

JING Yong-jun, born in 1977, MS, senior engineer, CCF member(43390M), his research interests include graph data mining and anomaly detection.



宋吉飞(1974 -),男,宁夏银川人,硕士,研究方向为网络安全和通信工程。**E-mail:**songjf@nxixp.cn

SONG Ji-fei, born in 1974, MS, his research interests include network security and communication engineering.